3D-LIP Gait Planning and Path Planning Based on Cassie Robot

Xia Yixuan, Zhou Jianxiang, Yang Boyu, Wei Jinqi, Yang Xin, Zhang Xipeng

Abstract: Cassie is a bipedal robot designed based on the joints that birds typically have. Based on Cassie's open-source URDF model available on GitHub, this paper first introduces the structure of the robot, and applies the 3D Linear Inverted Pendulum (3D-LIP) model to gait planning. The robot's ability to walk upright, climb stairs, and turn is demonstrated in the PyBullet simulation environment. Due to time constraints, the group was unable to complete a fully stable walking controller; therefore, path planning was implemented on the open-source model developed by Caltech. The group compared global and local path planning algorithms for mobile robots. After locating obstacles using computer vision, the Artificial Potential Field (APF) method was used to autonomously plan paths, and obstacle avoidance and pole-circumnavigation demonstrations were realized on the Gazebo platform (Ubuntu).

Keywords: Biped Robot; Linear Inverted Pendulum Model; Path Planning; Gait Design; Cassie

1 Research Motivation

Cassie is an ostrich-inspired bipedal robot invented and manufactured by Agility Robotics, a company incubated from Oregon State University. It received a $1 million grant from DARPA (Defense Advanced Research Projects Agency). The robot has 20 degrees of freedom across its body, with seven joints on each leg. Its nearly fully articulated leg structure enables versatile and free movement of the legs. The spring-loaded knees and ankles can absorb impacts, demonstrating the robot's agility. Using Cassie as a platform, researchers have proposed a wide variety of control strategies.

This paper is based on the open-source control model proposed by Caltech. It simplifies the model by cutting open the closed-loop linkage mechanism and adding constraints and angular relationships between the joints forming the linkage. The paper demonstrates motion planning under the simplified model, as well as obstacle localization and trajectory planning based on visual recognition. Section 2 describes the basic structure of the Cassie robot and the solution for forward and inverse kinematics, providing the theoretical foundation for subsequent simulations. Section 3 introduces 3D inverted pendulum gait planning, starting from the 2D Linear Inverted Pendulum model. Section 4 introduces the controller proposed by Caltech. Building on the preceding sections, Section 5 describes fundamental path planning methods, ultimately using computer vision for obstacle localization, then employing the Artificial Potential Field method for robot path planning, and completing a pole-circumnavigation demonstration. Finally, a summary of current problems and limitations is provided.

2 Model Construction

2.1 Basic Model

The Cassie robot is manufactured by Agility Robotics. The robot's form is inspired by the cassowary, a flightless bird native to New Guinea—those without a biology background may compare it to an ostrich. Cassie stands approximately 1 m tall and has a total mass of 31 kg, of which the battery located in the torso accounts for approximately 4 kg. As shown in Figure 1, the robot's floating-base model has 20 degrees of freedom. Each leg has seven joints, five of which are driven by motors, while the other two are passive joints realized through a four-bar linkage mechanism. The three actuated joints on each leg correspond to setting the leg's roll, yaw, and pitch angles relative to the torso; one motor sets the 'knee' angle; and the final motor adjusts the foot pitch angle.

The variables (qx, qy, qz) represent the Cartesian coordinates of the torso. qyaw, qpitch, qroll are Euler angles in z-y-x order. Joints q1, q2, q3 are the hip roll, yaw, and pitch angles respectively; q4 is the knee pitch angle; q5 is the shin pitch; q6 is the tarsus pitch; and q7 is the foot pitch angle.

For convenience, joints (q1, q2, q3, q4, q7) are the actuated joints. (q5, q6) are underactuated joints formed by introduced springs. When the springs are uncompressed, q5 and q6 satisfy the following relationship:

$q5 = 0$, and $q6 = -q4 + 13°$

For ease of subsequent modeling, the model is simplified. The linkage is assumed to be a virtual holonomic distance constraint acting between the hip and the heel spring connector:

$\eta_ach(q_l) := d(q_l) - 0.5012 = 0,$

It is also assumed that when one leg is in the swing phase, the spring on that leg is rigidly fixed:

$\eta_sw(q)^T := [q_s, q_hs]^T = 0.$

2.2 Kinematic Model

For the Cassie robot, regarding the selection of control variables h0, there are restrictive physical meanings between the actuated joints and the walking mode. Therefore, Caltech adopted a formulation that regulates the torso attitude angle, the stance foot and swing foot lengths, and the swing foot attitude angle.

When the right leg is in stance and the left leg is swinging, the forward kinematics for leg length and leg pitch are computed based on the robot's configuration.

3 Gait Planning Based on 3D-LIP

3.1 2D Linear Inverted Pendulum Model Analysis

Before applying the 3D Linear Inverted Pendulum model to Cassie robot gait planning, it is necessary to first consider the fundamental principles of bipedal dynamic walking planning under an ideal model — the 2D Linear Inverted Pendulum (LIP) model. The LIP model first approximates the robot and makes three assumptions: (1) all mass of the robot is concentrated at its center of mass (CoM); (2) the robot's leg is simplified as a telescoping massless rod that contacts the ground through a freely rotating pivot point; and (3) the robot's motion is constrained to the 2D plane determined by the vertical axis and the longitudinal direction of walking. Under these constraints, the Cassie robot can be simplified to a 2D linear inverted pendulum model, as shown in Figure 3.

The inputs to the inverted pendulum include a torque τ applied at the pivot point and a telescoping force f along the leg linkage direction. Assuming that the moment exerted by the ground on the foot during walking is negligible, i.e., $\tau = 0$, the telescoping force f can be decomposed into a vertical gravity compensation component and a horizontal acceleration component. Intuitively, the inverted pendulum maintains a constant CoM height by extending its leg length as it falls. This type of pendulum is called a Linear Inverted Pendulum.

After the vertical component of the leg telescoping force balances gravity, a horizontal component remains. This component accelerates the CoM horizontally. The corresponding equation of motion is:

$M\dot{x} = f \sin\theta = Mg \tan\theta = Mg/z \cdot x$

where x and z are the position coordinates of the inverted pendulum's CoM. Rearranging yields the differential equation for horizontal CoM motion:

$\ddot{x} = (g/z)\, x$

When z is constant, the horizontal kinematics are easily solved from this ODE:

$x(t) = x(0) \cosh(t/T_c) + T_c\, \dot{x}(0) \sinh(t/T_c)$

$\dot{x}(t) = x(0)/T^c \sinh(t/T^c) + \dot{x}(0) \cosh(t/T^c)$

$T^c \equiv \sqrt{(z/g)}$

where T^c is a constant depending on the CoM height and gravitational acceleration. x(0) and $\dot{x}(0)$ are the initial position and velocity of the CoM at time zero.

3.2 3D Linear Inverted Pendulum Model Analysis

The 3D robot is approximated as an inverted pendulum consisting of a point mass concentrating all the mass and a massless leg connecting this point mass to the support point, as shown in Figure 5. The torque at the support point is zero, so it can rotate freely. The leg can telescope under the action of telescoping force f, equivalent to the bending or extension of the robot's knee joint, causing up-and-down movement of the CoM. The telescoping force f can be decomposed into components along x, y, and z:

$fx = (x/r)f, \quad fy = (y/r)f, \quad fz = (z/r)f$

where r is the distance between the support point and the CoM. The CoM is only subject to the telescoping force and gravity, so its equations of motion are:

$M\ddot{x} = (x/r)f, \quad M\ddot{y} = (y/r)f, \quad M\ddot{z} = (z/r)f - Mg$

Similar to the 2D case, a constraint surface for the 3D inverted pendulum is defined as:

$z = kx \cdot x + ky \cdot y + zc$

where kx, ky determine the inclination of the plane, and zc determines its height. To keep the CoM moving on the constraint surface, its acceleration must be perpendicular to the surface normal. Solving for f:

$f = Mgr / zc$

By applying a telescoping force proportional to the leg length r, the CoM can be controlled to move on the constraint surface. The horizontal equations of motion are:

$\ddot{x} = (g/zc)\, x, \quad \ddot{y} = (g/zc)\, y$

It can be seen that the constraint surface parameters kx, ky do not affect the horizontal motion of the CoM. Such an inverted pendulum is called a 3D Linear Inverted Pendulum (3D-LIP). Its horizontal equations of motion are identical to those of the 2D LIP model.

3.3 Generation of 3D Walking Patterns

Walking Primitive

For 3D walking, the support foot must be switched simultaneously in both the x and y directions. In the following discussion, the support foot switching period is fixed, with each step's support time being T_sup.

In the time interval [0, T_sup], the 3D LIP trajectory is a hyperbola symmetric about the y-axis, referred to as a Walking Primitive. Given the support time T_sup and the z-intercept of the constraint plane, the walking primitive is uniquely determined by its terminal position $(\bar{x}, \bar{y})$ due to symmetry. The terminal velocity can be computed as follows.

Setting the initial conditions along the x-axis as $(-\bar{x}, \bar{v}x)$ and the terminal position as $\bar{x}$, according to the LIP analytical solution:

$\bar{x} = -\bar{x}\, C + Tc\, \bar{v}x\, S$

where:

$Tc = \sqrt{(zc/g)}, \quad C \equiv \cosh(T_sup/Tc), \quad S \equiv \sinh(T_sup/Tc)$

The terminal velocity is:

$\bar{v}x = \bar{x}\, (C+1)/(Tc\, S)$

Similarly, for the y-component of the walking primitive, the terminal position and velocity are:

$\bar{y} = \bar{y}\, C + Tc(-\bar{v}y)S$

$\bar{v}y = \bar{y}\, (C-1)/(Tc\, S)$

Walking Parameters

In many practical situations such as stair climbing or obstacle avoidance, it is necessary to directly specify foot placement positions, commonly described using step length and step width. The n-th foot placement position (px^(n), py^(n)) is expressed as:

$[px^{(n)}, py^{(n)}]^T = [px^{(n-1)} + sx^{(n)}, py^{(n-1)} - (-1)^n\, sy^{(n)}]^T$

This assumes the starting support foot is the right foot; if it is the left foot, substitute +(-1)^n for -(-1)^n. The walking parameters for the n-th step are determined by the step length and width of the (n+1)-th step:

$[\bar{x}^{(n)}, \bar{y}^{(n)}]^T = [sx^{(n+1)}/2, (-1)^n\, sy^{(n+1)}/2]^T$

The terminal velocity of the walking primitive is:

$[\bar{v}x^{(n)}, \bar{v}y^{(n)}]^T = [(C+1)/(Tc\, S)\, \bar{x}^{(n)}, (C-1)/(Tc\, S)\, \bar{y}^{(n)}]^T$

Foot Placement Adjustment

With fixed walking period and foot placement time, the walking speed can be controlled by adjusting the foot placement position. Qualitatively, placing the foot closer results in higher speed, while placing it further away results in lower speed.

The relationship between the adjusted foot placement p* and the final state of step n is given by the following equations. To compute the error between the resulting final state and the target state, an error evaluation function N is defined as:

$N \equiv a(x^d - xf^{(n)})^2 + b(\dot{x}^d - \dot{x}f^{(n)})^2$

where a, b are positive weighting factors. Setting $\partial N/\partial p^* = 0$, the foot placement that minimizes N is obtained.

Change of Walking Direction

As shown in Figure 9, direction changes during walking can be achieved by incorporating directional information into the foot placement determination. The n-th step's foot placement is updated with a rotation matrix based on the turning angle, and the walking primitive and velocity are updated accordingly.

From LIP Model to Multi-link Model

The simplest way to perform motion planning for an actual robot based on the LIP model is to have the robot's waist track the CoM motion of the inverted pendulum. First, the actual CoM position in the waist coordinate frame is obtained from the multi-link model. Then, assuming the relative position of the CoM in the waist remains unchanged, the waist position is obtained directly from the inverted pendulum. Finally, the swing foot trajectory is computed so that it reaches the desired landing point at the specified time. Once the waist and foot motions are determined, joint angles for each leg can be obtained through inverse kinematics.

3.4 Simulation Demonstration Based on 3D-LIP Gait Planning

Based on the above theoretical analysis, three demonstration tasks were completed based on 3D-LIP: straight walking, turning, and stair climbing. First, Python's matplotlib library was used to draw 3D gait

animation demonstrations in 3D space and the CoM trajectory projected onto the xy-plane. Then, in the PyBullet simulation environment, the CoM trajectory and foot placement trajectory were solved through inverse kinematics to obtain joint angles, which were then input into the Cassie model for position control.

Straight Walking: 3D gait animation was generated in matplotlib and corresponding CoM/foot trajectories were projected onto the xy-plane. The Cassie straight-walking gait was then demonstrated in PyBullet.

Turning: A 3D turning gait demonstration was generated in matplotlib. The Cassie turning gait was demonstrated in PyBullet.

Stair Climbing: A 3D stair-climbing gait demonstration was generated in matplotlib. The Cassie stair-climbing demonstration was completed in PyBullet.

4 Gait Planning Based on Full Order Model

For subsequent path planning tasks, the group adopted an open-source full-order model-based Cassie walking model developed by Caltech.

4.1 Full Order Model vs. Reduced Order Model

Although the 3D-LIP-based gait planning is computationally faster and more efficient, only considering the local behavior of the system limits the flexibility of overall locomotion and affects energy efficiency, and may require additional optimization to ensure feasible walking patterns. Additionally, using the LIP model faces challenges such as utilizing the robot's full capabilities, accounting for motor and joint constraints, and determining how to relate the full-order system state with the LIP motion.

However, an important feature of the LIP model is that the CoM velocity can be regulated based on foot placement — more precisely, by changing the angle at which the swing foot contacts the ground. In fact, the open-loop center of the LIP CoM velocity is an integrator in the longitudinal and lateral directions, and can therefore be stabilized with a proportional controller.

4.2 Virtual Constraints

Periodic gaits on Cassie are realized through virtual constraints, which are further 'gain-scheduled' based on the robot's current velocity to create continuously stable gaits. The resulting closed-loop system has the property that its CoM velocity approximately evolves as an integrator in the longitudinal and lateral directions, similar to the inverted pendulum model. Walking speed can be regulated to a desired value via 'foot placement position'.

Virtual constraints are functional relationships between the robot's generalized coordinates, used in system feedback control. They are defined as:

$y = h(q, \tau, \alpha) = h0(q) - hd(\tau, \alpha)$

where h0 is the physical quantity being regulated and hd is the desired value. The controller is designed to drive the virtual constraint output y to zero.

For the Cassie robot, given the restrictive physical meanings between actuated joints and walking modes, Caltech chose to regulate the torso attitude angle, stance and swing foot lengths, and swing foot attitude angle, as these physical quantities are directly related to gait behavior.

4.3 Gait Library

The open-source model controller from Caltech is based on a gait library. It allows the design of periodic walking gaits using a full-body dynamics model while respecting joint angle limits, ground reaction force ranges, and motor power constraints, and optimizes energy consumption per step at a given speed. The

desired value hd in the virtual constraints is obtained by linearly interpolating a discrete gait library, where each entry corresponds to a specific walking speed. The gait library is obtained by minimizing a cost function that includes velocity tracking error, torso pitch, torso roll, joint torques, and joint position constraints.

4.4 Controller

The controller framework consists of a feedback control loop composed of virtual constraints and the gait library to maintain stable walking. Overall, the closed-loop system behaves similarly to the LIP model in regulating CoM velocity — walking speed can be adjusted by modifying the swing leg pitch and roll angles.

5 Path Planning

5.1 Comparison of Path Planning Algorithms

The group primarily compared the local planning algorithm used — the Artificial Potential Field (APF) method — with the classical global planning graph-search algorithm, A*.

Path planning algorithms for mobile robots can broadly be categorized into global planning and local planning, primarily classified based on whether global information is known. In practice, many algorithms can serve as either global or local planners depending on design. Here, the group selected the A* algorithm, a classic graph-search global planner, as a comparison target.

A* Algorithm: A* is an effective direct search algorithm for finding the shortest path in a static road network. Its formula is:

$$f^*(n) = g^*(n) + h^*(n)$$

where f*(n) is the minimum cost estimate from the initial state through state n to the goal state; g*(n) is the minimum cost in state space from the initial state to state n; h*(n) is the minimum estimated cost of the path from state n to the goal state.

If h(n) ≤ actual distance from n to goal, A* can find the shortest path, but searches many nodes with a large search range and low efficiency. If h(n) > actual distance, fewer nodes are searched with higher efficiency, but the resulting path may not be shortest. The closer h(n) is to the actual distance from n to the goal, the better A* performs. When h(n) = 0, i.e., f(n) = g(n), A* degenerates to Dijkstra's algorithm. When h(n) >> g(n), A* degenerates to BFS.

A* is simple in principle and responds quickly to the environment, but has high computational cost, large memory usage, and the resulting path may not be optimal.

In comparison, the APF method moves the robot under the combined influence of an attractive field from the goal and repulsive fields from nearby obstacles. At each step, it finds the direction of the steepest descent in the potential field. Resulting paths are smooth, and the method is simple and easy to implement. However, it is prone to local minima and may fail to find a path along diagonal directions or when facing large obstacles.

The main APF formulas are the potential field function and gradient computation:

$$U(q) = U_att(q) + U_rep(q)$$

$$F(q) = -\nabla U(q)$$

Two solutions exist for the local optimum problem in APF: (1) adjusting the step size and other potential field parameters to avoid local minima, and (2) using local minimum detection functions to escape when the robot is trapped.

The group applied the APF algorithm with reference to wheeled robot motion planning. In practice, path planning for walking robots is more complex, since legged robots can step over or onto obstacles of

certain heights, or avoid them through leg trajectory planning. More complex algorithms exist for legged robot path planning, but due to knowledge limitations, the group used a wheeled robot path planning approach instead.

1) Visual Obstacle Detection, Localization, and Path Planning

After selecting the path planning algorithm, the group implemented visual obstacle detection and localization. Since the Cassie model does not carry a camera, a global camera was used to photograph the obstacles. Circular objects were chosen to represent obstacles, as they provide stronger symmetry when computing the potential field, simplifying the calculation.

Visual processing was performed using Python. Images were converted to grayscale, and OpenCV was used to extract feature contours and identify obstacle outlines and center points. Since grayscale images were used, it was necessary to handle the lighting/shadow effects produced by obstacles — by filtering shadows based on area and shape. This allowed localization and size estimation of the obstacles themselves.

The generated obstacle positions and sizes were then imported into the path planning code, with start and end positions set, and path planning was carried out.

5.2 Pole-Circumnavigation Demonstration and Path Simulation in Ubuntu + Gazebo

Since the 3D-LIP implemented in PyBullet lacked a motion controller and could not walk on the ground, the path planning algorithm used Caltech's Gazebo-based locomotion model. The simulation environment used was Ubuntu with Gazebo, and ROS was used for node communication to control the motion parameters of the model in Gazebo.

The group primarily modified parameters in /Cassie_interface_node, directly adjusting the joystick position that controls Cassie's movement, mapping it to the robot's velocity in the sagittal and coronal planes to further control the robot's motion trajectory.

To verify the control logic, the group implemented a simple pole-circumnavigation model, making Cassie walk a sinusoidal path in the simulation environment: coronal-plane speed using a cosine function and sagittal-plane speed constant.

After verifying the control logic, the path planning data was exported as directional segments for motion. The code was designed to move in one of eight directions at each step. The directions were exported as combinations of -1, 0, and 1 representing eight directions and imported into the C++ Cassie_interface_node.cpp file. The file reads each segment's ROS::Time motion speed in order and outputs it to the corresponding subscriber, ultimately mapping to Cassie's sagittal and coronal velocities in Gazebo.

The actual motion path in Gazebo closely reproduced the planned path. Although there may be slight deviations between the actual Gazebo path and the planned path — because the joystick values rather than velocities are directly controlled, and the controller applies some interpolation/smoothing to the output — the overall trajectory remains largely consistent. The simulation did not use Caltech's QP controller, yet the walking gait was relatively stable and the path could be maintained broadly consistent.

6 Collaboration and Contributions

- 22% — Xia Yixuan: Path planning, Gazebo motion simulation and walking trajectory planning, PPT production, Team leader
- 22% — Zhou Jianxiang: 3D-LIP, PyBullet simulation, PPT production
- 20% — Yang Boyu: 3D-LIP, PyBullet simulation, PPT production
- 12% — Zhang Xipeng: 3D-LIP

- 12% — Yang Xin: Analysis and comparison of path planning algorithms, PPT production
- 12% — Wei Jinqi: Environment setup, obstacle visual detection, path planning

7 Challenges and Solutions

Challenges

1. Gazebo model simulation takes a long time, and significant differences exist between Linux and Windows in file encoding and program execution. Virtual machines and dual-boot setups limit Ubuntu's performance due to hardware constraints.
2. The Gazebo model uses joystick control with built-in speed smoothing, requiring time to adjust. This limits how quickly speed can be switched and may cause deviations from the planned path.
3. Setting up the Ubuntu simulation environment and running Caltech's open-source simulation involved tedious environment configuration, which affected project progress.
4. In the simulation environment, the Cassie model's foot-to-ground contact is line-based, making it difficult to maintain stability during dynamic walking. In the 3D-LIP-based gait planning, due to knowledge limitations, it was difficult to build a stable walking controller to track the desired trajectory without falling.

Solutions

5. When possible, use an independent system and program directly in a single OS environment.
6. Choose appropriate speed update intervals and ratios to make the overall motion more coordinated; alternatively, directly modify controller parameters to bypass joystick control.
7. In the 3D-LIP gait planning demonstration, position control (rather than torque control) was used for each joint to ensure Cassie follows the expected trajectory in simulation. Additionally, the Cassie base was constrained in degrees of freedom to prevent it from tipping over.

8 Conclusion

Based on the open-source Cassie robot URDF model, this paper first described the degrees of freedom of the robot joints and the overall mechanical structure, then introduced the control method based on the Linear Inverted Pendulum model, demonstrating the robot's ability to walk upright, climb stairs, and turn in PyBullet. Additionally, path planning was carried out on the Gazebo platform using Caltech's controller. By comparing different algorithms, the optimal strategy was identified. After visually locating obstacles, the Artificial Potential Field method was used to autonomously plan paths.

References

[1] https://zhuanlan.zhihu.com/p/385733813

[2] https://www.cs.cmu.edu/~motionplanning/lecture/Chap4-Potential-Field_howie.pdf

[3] Reher, Jenna and Aaron D. Ames. 'Inverse Dynamics Control of Compliant Hybrid Zero Dynamic Walking.' Submitted to 2021 IEEE ICRA and Robotics and Automation Letters (RA-L).

[4] Reher, Jenna, Wen-Loong Ma, and Aaron D. Ames. 'Dynamic walking with compliance on a Cassie bipedal robot.' 2019 18th European Control Conference (ECC). IEEE, 2019.

[5] Gong, Y., Hartley, R., X Da, Hereid, A., Harib, O., & Huang, J. K., et al. (2018). Feedback control of a cassie bipedal robot: walking, standing, and riding a segway.

[6] Open-sourced repository for the C++ controller code: https://github.com/jpreher/cassie_documentation

[7] https://www.researchgate.net/publication/298659076_Introduction_to_Humanoid_Robotics

[8] Dalibard, S., El Khoury, A., Lamiraux, F., Nakhaei, A., Taïx, M., and Laumond, J.-P. Dynamic walking and whole-body motion planning for humanoid robots: an integrated approach. The International Journal of Robotics Research, 32(9-10):1089-1103, 2013.

[9] Da, X., Harib, O., Hartley, R., Griffin, B., and Grizzle, J. W. From 2D design of underactuated bipedal gaits to 3D implementation: Walking with speed tracking. IEEE Access, 4:3469-3478, 2016.

[10] Canudas-de-Wit, C. On the concept of virtual constraints as a tool for walking robot control and balancing. Annual Reviews in Control, 28(2):157-166, 2004.